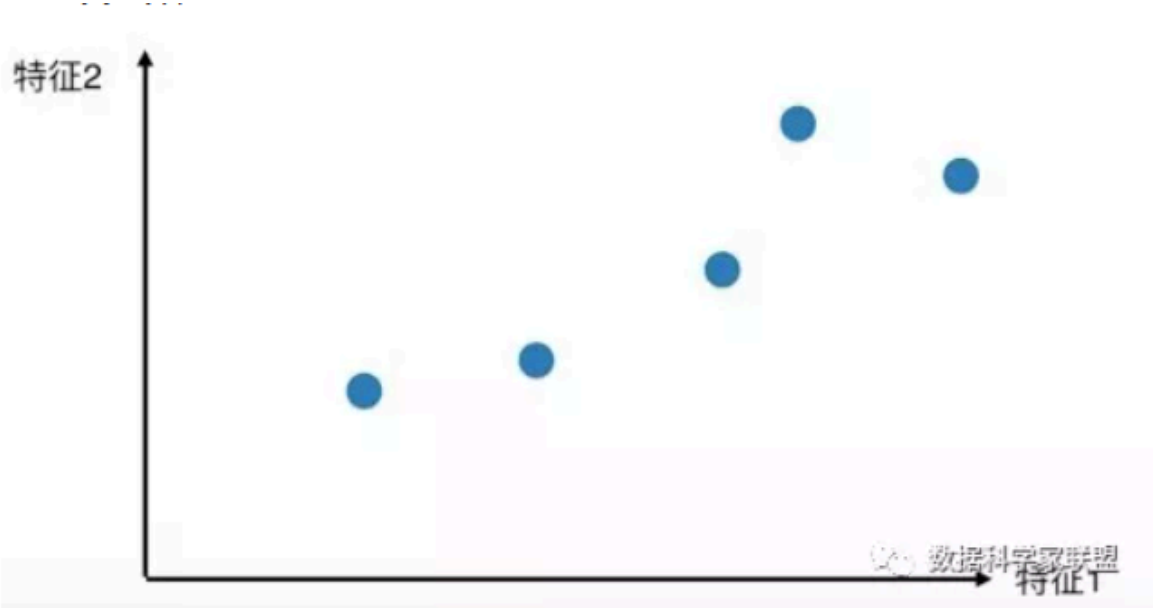


第三次大作业报告

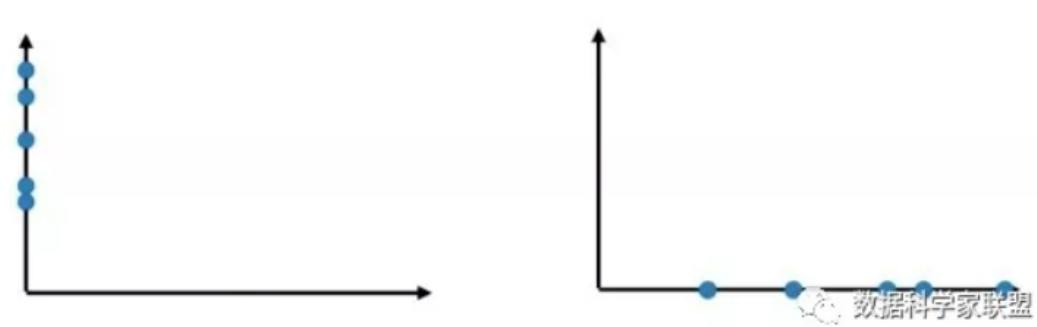
实验原理

PCA降维

PCA(Principal Component Analysis)，即主成分分析方法，是一种使用最广泛的数据降维算法。它的主要思想是将 n 维特征映射到 k 维上，这 k 维是全新的正交特征也被称为主成分，是在原有 n 维特征的基础上重新构造出来的 k 维特征。它能够有效缓解维度灾难。



例如，我们根据数据的两个特征画出散点图，并将二维的数据降为一维。此时，我们有两种降维的方法，它们的效果如下：



从这张图上可以看出，保留 x 特征更能体现数据之间的差异。因此，我们会保留 x 特征而放弃 y 特征。

在实际的操作中，PCA降维的步骤通常如下：

1. 将原始数据按列组成 n 行 m 列矩阵 A 。
2. 将 X 的每一行进行零均值化。
3. 求出协方差矩阵及其特征值与特征向量。
4. 将特征向量按特征值大小排序，取前 x 行组成新矩阵 B 。
5. 矩阵 $C = BA$ 就是降维后的数据。

决策树

决策树是一种解决分类问题的算法，它采用树形结构，使用层层推理来实现最终的分类。

决策树算法中一个非常重要的概念是信息增益。信息增益 $g(D, A)$ ，定义为集合D的熵 $H(D)$ 与特征A给定条件下D的条件熵 $H(D|A)$ 之差

决策树算法主要有三个步骤：

1. 特征选择：在训练数据集中，每个样本的属性可能有很多个，不同属性的作用有大有小。因而特征选择的作用就是筛选出跟分类结果相关性较高的特征，也就是分类能力较强的特征。
2. 决策树生成：选择好特征后，就从根节点出发，对节点计算所有特征的信息增益。选择信息增益最大的特征作为节点特征，根据该特征的不同取值建立子节点；对每个子节点使用相同的方式生成新的子节点，直到信息增益很小或者没有特征可以选择为止。
3. 决策树剪枝：剪枝的主要目的是对抗过拟合，通过主动去掉部分分支来降低过拟合的风险。

主流的决策树算法有这三种：

1. ID3算法：最早的决策树算法，使用信息增益选择特征。
2. C4.5算法：使用信息增益比来选择特征。
3. CART算法：使用基尼系数取代了信息熵模型。

实验过程和结果

PCA降维

```

from ucimlrepo import fetch_ucirepo
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# fetch dataset
wine_quality = fetch_ucirepo(id=186)

# data (as pandas dataframes)
X = wine_quality.data.features
y = wine_quality.data.targets

# metadata
print(wine_quality.metadata)

# variable information
print(wine_quality.variables)

# 标准化处理
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(pd.DataFrame(X_scaled, columns=X.columns).head())

```

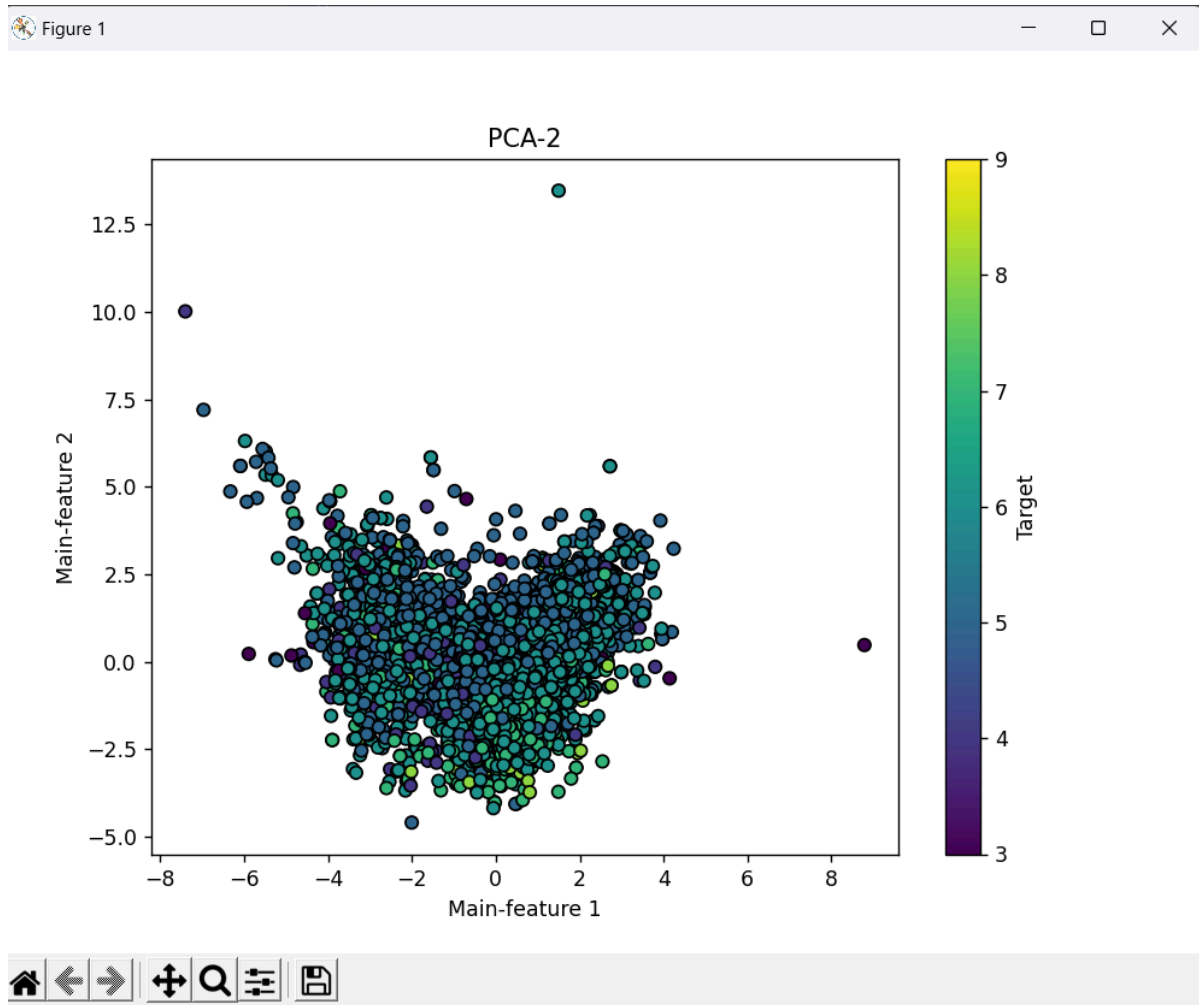
数据集的引入和数据的标准化处理。

```

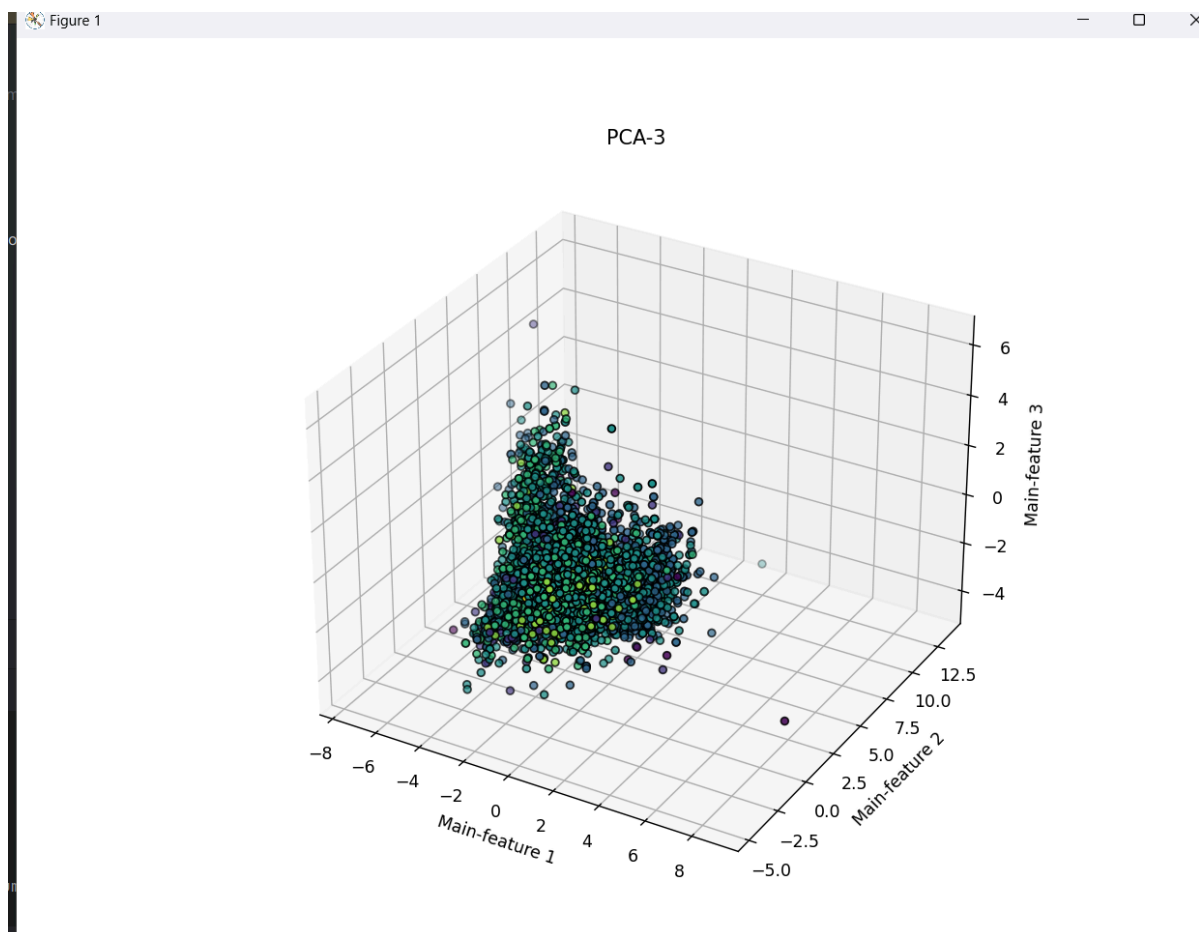
# 维度
n_components = 2
# PCA降维
pca = PCA(n_components=n_components)
X_pca = pca.fit_transform(X_scaled)

```

使用PCA算法对数据进行降维。



两个主成分的降维结果。



三个主成分的降维结果。

五个主成分的降维结果维度太高，不能可视化。

从图中可以看出，三个主成分的降维结果比两个主成分的降维结果更能表现数据之间的差异。有些点在二维平面上几乎重合，但在三维空间里有不同的z值。

决策树分类器

```
# fetch dataset
wine_quality = fetch_ucirepo(id=186)

# data (as pandas dataframes)
X = wine_quality.data.features
y = wine_quality.data.targets

# metadata
print(wine_quality.metadata)

# variable information
print(wine_quality.variables)

X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42)
```

数据集的引入，训练集和测试集的划分。

```
elif tree == 1:
    id3_model = Id3Estimator()
    y_train = y_train.values.flatten()
    id3_model.fit(X_train, y_train)

    # 预测和评估
    y_pred_id3 = id3_model.predict(X_test)
    print("ID3 Accuracy:", accuracy_score(y_test, y_pred_id3))
    print("ID3 Confusion Matrix:\n", confusion_matrix(y_test, y_pred_id3))

elif tree == 2:
```

使用最早的ID3模型。

```
[13 rows x 7 columns]
ID3 Accuracy: 0.5317948717948718
ID3 Confusion Matrix:
[[ 0  2  3  2  2  0  0]
 [ 3  8 30 25  2  1  0]
 [ 6 17 357 207 22  4  0]
 [ 2 11 232 515 124 10  0]
 [ 0  3  27 128 148  9  0]
 [ 0  1  1  21  17  9  0]
 [ 0  0  0  0  1  0  0]]

Process finished with exit code 0
```

分类准确度为53%。

```

elif tree == 2:
    c45_model = C45Classifier()
    c45_model.fit(X_train, y_train)

    # 预测和评估
    y_pred_c45 = c45_model.predict(X_test)
    print("C4.5 Accuracy:", accuracy_score(y_test, y_pred_c45))
    print("C4.5 Confusion Matrix:\n", confusion_matrix(y_test, y_pred_c45))

```

使用C4.5模型。

```

[13 rows x 7 columns]
C4.5 Accuracy: 0.4564102564102564
C4.5 Confusion Matrix:
[[ 0  1  1  2  3  2  0]
 [ 0  6  8 15 30 10  0]
 [ 0  3 206 149 197 58  0]
 [ 1  2  53 467 269 99  3]
 [ 1  2  18  69 192 31  2]
 [ 0  0   3  10  17 19  0]
 [ 0  0   0   0   0  1  0]]

```

分类准确度为46%。

```

if tree == 0:
    cart_model = DecisionTreeClassifier(criterion='gini') # 使用 Gini 不纯度
    cart_model.fit(X_train, y_train)

    # 预测和评估
    y_pred_cart = cart_model.predict(X_test)
    print("CART Accuracy:", accuracy_score(y_test, y_pred_cart))
    print("CART Confusion Matrix:\n", confusion_matrix(y_test, y_pred_cart))
elif tree == 1:

```

使用CART模型。

```
[13 rows x 7 columns]
CART Accuracy: 0.5753846153846154
CART Confusion Matrix:
[[ 0  1  2  5  1  0  0]
 [ 3 17 27 21  0  1  0]
 [ 1 17 385 170 33  6  1]
 [ 3 27 212 532 96 24  0]
 [ 1  5  20  99 170 20  0]
 [ 0  0  2  17 12 18  0]
 [ 0  0  0  1  0  0  0]]

Process finished with exit code 0
```

分类准确度为58%。

接下来对三种模型的参数进行调整。在这里使用了网格搜索的办法。

网格搜索是指在一个包含待优化超参数的搜索空间中找到一组最佳的超参数组合，通过穷举搜索所有给定参数的可能组合，自动地评估每一组超参数配置并选择性能最好的配置。

调整的参数有：

1. max_depth: 树的最大深度。
2. min_samples_split: 表示一个节点分裂所需的最小样本数。
3. min_samples_leaf: 叶节点的最小样本数。
4. max_features: 选择分裂时考虑的最大特征数。

调整CART模型，ID3模型的参数。经过调参后，模型的分类准确率都上升了。

CART模型：

```
if tree == 0:
    cart_model = DecisionTreeClassifier(criterion='gini')
    param_grid = {
        'max_depth': [10, 20, 30, None],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 4],
        'max_features': ['auto', 'sqrt', 'log2', None]
    }
    grid_search = GridSearchCV(cart_model, param_grid, cv=4, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    print("Best Parameters for CART:", grid_search.best_params_)
    cart_model = grid_search.best_estimator_
    y_pred_cart = cart_model.predict(X_test)
    print("CART Accuracy:", accuracy_score(y_test, y_pred_cart))
    print("CART Confusion Matrix:\n", confusion_matrix(y_test, y_pred_cart))
```

```
[13 rows x 7 columns]
Best Parameters for CART: {'max_depth': 30, 'max_features': 'auto', 'min_samples_leaf': 1, 'min_samples_split': 2}
CART Accuracy: 0.5974358974358974
CART Confusion Matrix:
[[ 0  1  1  6  0  1  0]
 [ 3 13 32 15  4  2  0]
 [ 1 17 403 154 31  7  0]
 [ 1 20 193 560 101 19  0]
 [ 0  5  22  99 170 19  0]
 [ 0  0  2  13  15 19  0]
 [ 0  0  0  1  0  0  0]]

Process finished with exit code 0
```

```
Execution Time: 5.3655 seconds
```

ID3模型:

```
elif tree == 1:
    id3_model = Id3Estimator()
    y_train = y_train.values.flatten()
    param_grid = {
        'max_depth': [10, 20, 30, None],
        'min_samples_split': [2, 5, 10]
    }

    grid_search = GridSearchCV(id3_model, param_grid, cv=5, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    print("Best Parameters for ID3:", grid_search.best_params_)
    id3_model = grid_search.best_estimator_
    y_pred_id3 = id3_model.predict(X_test)
    print("ID3 Accuracy:", accuracy_score(y_test, y_pred_id3))
    print("ID3 Confusion Matrix:\n", confusion_matrix(y_test, y_pred_id3))
```

```
[13 rows x 7 columns]
Best Parameters for ID3: {'max_depth': 20, 'min_samples_split': 2}
ID3 Accuracy: 0.5317948717948718
ID3 Confusion Matrix:
[[ 0  2  3  2  2  0  0]
 [ 3  8 30 25  2  1  0]
 [ 6 17 357 207 22  4  0]
 [ 2 11 232 515 124 10  0]
 [ 0  3  27 128 148  9  0]
 [ 0  1  1  21  17  9  0]
 [ 0  0  0  0  1  0  0]]

Execution Time: 54.8581 seconds
```

关于决策树的讨论

1. 不同策略的决策树在性能上的差异: 在本实验中, CART模型的分类准确率高于ID3模型, 高于C4.5模型。ID3模型的执行时间高于CART模型。
2. 各策略的优缺点和适用场景:
 - ID3模型是最早的决策树模型, 有很多不足之处。例如, 它只能处理离散特征, 对于缺失值的情况没有考虑, 也没有考虑过拟合的问题。
 - C4.5模型可以处理连续的特征和缺失值, 但由于使用了熵模型, 算法效率不高。

- CART模型使用了基尼系数代替增益比，对模型进行了简化，算法效率高。
- 由于这三种算法在特征选择、处理连续特征和缺失值、停止条件这三个方面有所不同，因此在同样的数据集中，它们可能产生不同的决策树和预测结果。

PCA和决策树结合

```
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.3, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)
# 维度
n_components = 11
# PCA降维
pca = PCA(n_components=n_components)
X_pca_train = pca.fit_transform(X_train_scaled)
X_pca_test = pca.fit_transform(X_test_scaled)
```

首先使用PCA对数据进行降维。

```
start_time = time.time()
cart_model = DecisionTreeClassifier(criterion='gini')
param_grid = {
    'max_depth': [20, 30, 40, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['auto', 'sqrt', 'log2', None]
}
grid_search = GridSearchCV(cart_model, param_grid, cv=4, scoring='accuracy')
grid_search.fit(X_pca_train, y_train)
print("Best Parameters for CART:", grid_search.best_params_)
cart_model = grid_search.best_estimator_
y_pred_cart = cart_model.predict(X_pca_test)
end_time = time.time()
print("CART Accuracy:", accuracy_score(y_test, y_pred_cart))
print("CART Confusion Matrix:\n", confusion_matrix(y_test, y_pred_cart))
execution_time = end_time - start_time
print(f"Execution Time: {execution_time:.4f} seconds")
```

随后使用优化过的CART模型生成决策树。

不同降维数量下的决策树性能如下所示：

```
[13 rows x 7 columns]
Best Parameters for CART: {'max_depth': None, 'max_features': None, 'min_samples_leaf': 1, 'min_samples_split': 2}
CART Accuracy: 0.3958974358974359
CART Confusion Matrix:
[[ 0  0  7  0  1  1  0]
 [ 0  7 33 21  8  0  0]
 [ 5 32 300 230 37  9  0]
 [ 8 49 279 373 156 29  0]
 [ 1 11  61 132  87 23  0]
 [ 0  1  14  14  15  5  0]
 [ 0  0  0  0  0  1  0]]
Execution Time: 4.5467 seconds

Process finished with exit code 0
```

这是五个主成分的决策树性能。

```
[13 rows x 7 columns]
Best Parameters for CART: {'max_depth': 40, 'max_features': 'auto', 'min_samples_leaf': 1, 'min_samples_split': 2}
CART Accuracy: 0.38153846153846155
CART Confusion Matrix:
[[ 0  2  4  3  0  0  0]
 [ 4  4 33 18  8  1  1]
 [15 39 332 181 40  6  0]
 [15 50 380 338 98 13  0]
 [ 3 12  82 135 68 15  0]
 [ 0  5 12 21  9  2  0]
 [ 0  0  0  0  1  0  0]]
Execution Time: 3.1116 seconds
```

三个主成分的决策树性能。

```
[13 rows x 7 columns]
Best Parameters for CART: {'max_depth': None, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 2}
CART Accuracy: 0.35333333333333333
CART Confusion Matrix:
[[ 0  0  6  2  1  0  0]
 [ 0  1 31 25  8  4  0]
 [ 1 33 259 249 60 11  0]
 [ 9 44 322 376 114 29  0]
 [ 0 11  89 141 51 20  3]
 [ 0  5 11 21 10  2  0]
 [ 0  0  0  1  0  0  0]]
Execution Time: 2.8238 seconds
```

两个主成分的决策树性能。

从上面的实验可以看出，随着主成分数量减少，决策树分类准确率下降。

关于PCA降维与决策树分类性能的讨论

1. PCA降维对决策树分类性能的影响：PCA降维可以减少模型过拟合的风险，降低计算成本。但是，降维也会丢失一定的信息，有可能导致模型性能下降。在实践中，如果数据集中的特征相关性很强且存在大量冗余，PCA可能显著提升决策树的性能；相反地，如果原始数据集中的特征不存在很多冗余，包含较多的非线性信息，那么PCA可能会导致决策树分类准确率明显下降。
2. 探讨在降维后，信息损失与模型复杂度之间的权衡：降维会对模型复杂度产生直接的影响。当数据特征维度过高时，PCA降维可以有效降低模型复杂度，同时增加模型的性能。但是，如果降维过度，有用的信息可能会丢失，模型会变得过于简单，无法准确分类。在实践中，可以采用交叉验证来找到合适的降维方案。